Evaluating Leakage–Performance Tradeoffs in Encrypted Databases: An Experimental Study of Path ORAM and SEAL

Anuj Lall

Abstract

This project evaluates the security–performance tradeoffs of Path ORAM and the SEAL adjustable-leakage extension for encrypted database systems. Path ORAM provides strong protection against access-pattern leakage but incurs significant performance overhead due to the need to access full paths in an ORAM tree for each query. SEAL improves efficiency by partitioning the database into multiple smaller ORAM instances, controlled by a leakage parameter α\alphaα, which intentionally reveals limited information about data access patterns.

We implement both systems and evaluate them under several query workloads using synthetic datasets with skewed value distributions. Two leakage-abuse attacks are implemented and evaluated using the metrics Query Recovery Success Rate (QRSR) and Database Recovery Success Rate (DRSR). Experiments measure how attack success varies with the SEAL leakage parameter, query workload distribution, padding-based mitigations, and session-reset strategies.

The results show that increasing α\alphaα significantly reduces bandwidth per access, confirming the performance advantages of SEAL. However, the additional leakage introduced by partition identifiers has limited impact on attack success compared to response volume leakage. Workload skew has a much larger effect, with highly skewed workloads enabling near-perfect query recovery. Mitigation techniques such as response padding reduce attack effectiveness but introduce substantial bandwidth overhead.

Overall, the experiments highlight the complex tradeoffs between performance and security in encrypted database systems and demonstrate how workload characteristics and response-volume leakage strongly influence the practical effectiveness of inference attacks.

---

1. Introduction

Access-pattern leakage remains a major challenge in encrypted data systems. While encryption protects the contents of stored data, it does not hide which records are accessed or how frequently they are queried. An observer who can monitor access patterns may still infer sensitive information even when all data is encrypted. For example, repeated access to certain records or consistent query volumes can reveal trends, popular values, or relationships within the underlying dataset. As a result, protecting access patterns has become an important problem in secure data storage and encrypted database systems.

Oblivious RAM (ORAM) was introduced to address this problem by ensuring that sequences of data accesses appear indistinguishable to an external observer. Path ORAM is one of the simplest and most widely studied ORAM constructions. It organizes server-side storage as a tree of buckets and hides the true access pattern by reading and rewriting an entire root-to-leaf path for every logical access. Through the use of a client-side position map and a temporary

stash, Path ORAM ensures that the server cannot determine which specific data block was accessed. However, this strong security guarantee comes with significant overhead, since every operation requires moving multiple encrypted blocks regardless of the actual data requested.

To improve efficiency, later work proposed relaxing the strict security guarantees of ORAM in exchange for better performance. One such approach is SEAL, which introduces adjustable leakage into the system. Instead of storing all data within a single ORAM structure, SEAL partitions the database into multiple smaller ORAMs. A secret permutation is used to map logical identifiers to specific partitions, and the number of partitions is controlled by a parameter α. Increasing α reduces the size of each ORAM and therefore improves performance, but it also leaks limited information about which partition was accessed. This design creates a tunable security–performance tradeoff: larger values of α improve efficiency while potentially revealing more information to an observer.

This project investigates how this tradeoff behaves in practice by implementing both Path ORAM and SEAL and evaluating their behavior under several experimental conditions. In addition to measuring performance as α varies, we study how much information an attacker can recover from the leakage exposed by SEAL. Following the leakage-abuse framework proposed in the SEAL literature, we implement two attacks: a Query Recovery Success Rate (QRSR) attack that attempts to infer which queries were executed based on response volume, and a Database Recovery Success Rate (DRSR) attack that attempts to map encrypted tuples to plaintext values using both volume information and leaked partition identifiers.

Beyond the basic comparison between Path ORAM and SEAL, this work also explores several factors that may influence practical leakage. First, we evaluate how different access patterns affect attack success by comparing uniform workloads with skewed and hot-set access distributions. Second, we analyze how attacker confidence evolves over time by measuring recovery success at multiple observation checkpoints. Third, we evaluate padding as a mitigation strategy by hiding response volume information at the cost of increased overhead. Finally, we introduce a session-reset experiment to study whether short-lived observation windows limit the amount of leakage that an attacker can exploit.

The main contributions of this work are as follows:

- We implement Path ORAM and SEAL with a configurable leakage parameter α.
- We implement leakage-abuse attacks and evaluate their effectiveness using QRSR and DRSR metrics.
- We measure the security–performance tradeoff across different α values.
- We extend the experimental evaluation to include workload patterns, attacker confidence over time, padding mitigation, and session-length effects.

Together, these experiments provide a practical view of how adjustable leakage affects both efficiency and security in ORAM-based systems, and illustrate how real-world factors such as workload structure and observation time can influence the impact of access-pattern leakage.

2. System Design and Implementation

This section describes the design and implementation of the experimental system used to evaluate access-pattern leakage in Path ORAM and SEAL. The system was implemented in Python and organized into separate modules for storage logic, attack analysis, and experiment execution. The implementation was structured to keep the core ORAM functionality independent from the attack and evaluation layers, allowing experiments to be conducted without modifying the underlying storage mechanisms.

2.1 Path ORAM Implementation

The baseline system is an implementation of Path ORAM written in Python. The storage model follows the standard Path ORAM design in which server-side data is organized as a binary tree of buckets, while the client maintains metadata required to locate and manage blocks.

The server stores data as a full binary tree where each node contains a fixed-size bucket. Each bucket has a capacity of Z blocks, and all buckets are padded to this size using dummy blocks. Each block contains a logical identifier, a payload value, a leaf label indicating the block's assigned path, and a flag identifying whether the block is a real block or a dummy filler. Dummy blocks ensure that every bucket read or write operation transfers a constant amount of data, preventing the server from inferring information from bucket sizes.

The client maintains two key data structures: a position map and a stash. The position map stores the currently assigned leaf for every logical block identifier and acts as the client's secret routing state. The stash is a temporary buffer used during accesses to hold blocks that have been read from the tree but cannot immediately be placed back into buckets. The stash ensures correctness during eviction and allows blocks to be repositioned within the tree.

Each logical access is processed through a sequence of steps. First, the client retrieves the leaf currently associated with the requested block from the position map. Before performing the access, the block is immediately assigned a new random leaf and the position map is updated to reflect this reassignment. The client then reads the entire root-to-leaf path corresponding to the old leaf from the server. All non-dummy blocks encountered along this path are added to the stash.

Once the path is loaded, the client locates the target block in the stash. If the block does not yet exist, it is lazily created with a default value. The requested read or write operation is then applied to the block's data. After the logical operation is complete, the client rebuilds the accessed path by placing blocks from the stash back into buckets. Buckets are filled from the deepest nodes upward so that blocks are placed as deep as possible along the path consistent with their assigned leaves. Any remaining empty bucket slots are filled with dummy blocks.

Blocks that cannot yet be placed remain in the stash for future accesses. Finally, the reconstructed path is written back to the server.

Correctness of the Path ORAM implementation was verified using randomized tests comparing ORAM operations with a standard dictionary storing the same logical data. Additional invariant checks ensured that each bucket always contained exactly *Z* entries, that no duplicate block identifiers existed across the stash and server storage, and that each access touched exactly one root-to-leaf path. Stash size was also monitored to detect incorrect eviction behavior.

2.2 SEAL Implementation

SEAL was implemented as a thin wrapper around the Path ORAM implementation. Rather than modifying the internal ORAM logic, SEAL partitions the logical database into multiple smaller ORAM instances and routes accesses among them.

Given a logical database of size *n* and leakage parameter α, the system creates 2^α independent Path ORAM instances. Each sub-ORAM stores *n* / 2^α logical items. This partitioning reduces the size of each ORAM tree and therefore lowers the cost of individual accesses.

To determine which sub-ORAM handles a given logical block identifier, SEAL applies a conceptual pseudorandom permutation (PRP) to the identifier space. In the implementation, this permutation is modeled using a simple affine mapping over the identifier domain. For a logical identifier *i*, the permutation produces a value *j*, which is then split into two components: the most significant α bits determine the sub-ORAM index, while the remaining bits determine the block's local identifier within that ORAM.

As a result, each logical access is translated into a pair (*oram_index*,*local_id).* The SEAL wrapper routes the request to the corresponding Path ORAM instance and performs the access using the existing ORAM interface. This design preserves the correctness and security properties of the underlying ORAM while exposing limited information about which partition was accessed.

The parameter α is configurable and directly controls the tradeoff between performance and leakage. Larger values of α increase the number of sub-ORAMs, reducing per-access overhead but revealing more information about which partition was accessed. When α = 0, the system reduces to a single ORAM instance and behaves identically to standard Path ORAM.

2.3 Attack Implementation

The leakage-abuse attacks were implemented as experiment-layer analyses that operate on the observable metadata produced during queries. These attacks do not modify the ORAM or SEAL storage logic and instead analyze the information that would be visible to an external observer.

The attacker is assumed to observe two forms of leakage for each query: the size of the returned response (which may be padded) and the α-bit partition identifiers associated with returned tuples in the SEAL system. The attacker is also assumed to know the distribution of plaintext values in the dataset, representing a strong adversarial model.

Two attack metrics are evaluated. The Query Recovery Success Rate (QRSR) measures how often the attacker correctly infers the plaintext query value. The attack groups plaintext values by their known frequency in the dataset and compares these frequencies with the observed response volume. When multiple candidate values share the same volume, the attacker selects one randomly. QRSR is computed as the fraction of queries whose plaintext value is correctly guessed.

The Database Recovery Success Rate (DRSR) evaluates how accurately the attacker can map encrypted tuples to plaintext values. After estimating the query value using the same volume-based inference used in QRSR, the attacker uses the leaked partition identifiers to associate returned tuples with candidate plaintext values. Because multiple tuples may share the same identifier prefix, random tie-breaking is used when assigning tuples. DRSR is computed as the fraction of returned tuples that are correctly labeled.

2.4 Padding Mitigation

To reduce information leakage from response volume, a padding mitigation mechanism was implemented at the experiment layer. Padding was intentionally applied outside of the ORAM storage logic so that the underlying system remained unchanged and mitigation strategies could be evaluated independently.

For each query returning s real results, the observed response size was padded to the next power of a configurable base x. For example, when $x = 2$, response sizes are rounded up to the nearest power of two. If padding increases the response size, dummy identifiers are added to the observed response so that the attacker sees only the padded size rather than the true number of results.

This strategy reduces the uniqueness of response sizes and therefore weakens attacks that rely on volume information. However, padding also increases the amount of data transferred, introducing additional bandwidth overhead.

2.5 Experiment-Layer Extensions

Several extensions were implemented in the experiment layer to explore how leakage behaves under more realistic conditions. These extensions do not modify the ORAM or SEAL implementations but instead control the workloads and evaluation procedures.

First, the system supports multiple workload patterns. Uniform workloads select query values uniformly at random from the dataset and serve as a baseline. Zipf-like workloads introduce

skew by favoring a small subset of frequently accessed values. Hot-set workloads restrict queries to a fixed subset of values representing a working set that is repeatedly accessed.

Second, attacker success was evaluated over time by measuring QRSR and DRSR at multiple observation checkpoints. Instead of evaluating attacks only at the end of a run, the experiment framework re-evaluates the attacker after observing increasing prefixes of the query stream, allowing leakage accumulation to be studied over time.

Finally, a session-reset experiment was introduced to simulate short-lived observation windows. The workload is divided into multiple sessions of fixed length, and the attacker's internal state is reset between sessions while the dataset and ORAM configuration remain unchanged. Attack success is then computed independently for each session and aggregated across sessions. This experiment models scenarios in which an attacker observes only limited interaction windows rather than a continuous trace.

Together, these components form a modular evaluation framework consisting of a storage layer (Path ORAM and SEAL), an attack analysis layer, a mitigation mechanism, and an experiment layer for generating workloads and collecting measurements. This separation allowed the underlying storage implementation to remain stable while enabling a wide range of experimental analyses.

---

3. Experimental Setup

This section describes the environment, datasets, workloads, and evaluation methodology used to measure both the performance and security properties of the implemented systems. The goal of the experimental setup is to allow the security–performance tradeoffs of Path ORAM and SEAL to be evaluated in a controlled and reproducible manner.

3.1 Implementation Environment

All systems and experiments were implemented in Python 3 and executed locally within a virtual environment. The project was organized into modular components separating storage logic, attack analysis, and experiment execution. The Path ORAM implementation resides in `src/path_oram/`, while the SEAL wrapper is implemented in `src/seal/`. Attack implementations and mitigation utilities are located in `src/attacks/`, and the experiment framework—including workload generation, evaluation scripts, and plotting utilities—is contained in `src/eval/` and `src/workload/`.

Experiments were executed using Python runner scripts that load configuration files specifying parameter values and experiment types. Each run produces a directory containing the results and plots generated during the experiment, along with a snapshot of the configuration used. The

experiment framework relies on several common Python libraries. In particular, NumPy is used for synthetic dataset generation and sampling, and Matplotlib is used to generate plots for analysis and reporting. Additional functionality such as randomness, timing, and file handling uses Python's standard libraries.

3.2 Dataset Generation

Experiments were conducted using a synthetic dataset designed to approximate realistic query workloads while allowing precise control over value distributions. The target dataset size follows the project specification of n = 2^20 records (1,048,576 items). Each record corresponds to a logical encrypted tuple stored in the ORAM system and is associated with a plaintext attribute value used to define query results.

Dataset values were generated using a Zipf-like distribution, which produces a skewed frequency distribution in which some values occur much more frequently than others. This type of distribution is commonly used to model real-world access patterns such as keyword searches or database queries, where a small number of values are much more popular than the rest.

After generating the dataset, an inverted index was constructed mapping each plaintext value to the list of record identifiers containing that value. This index is used by the experiment framework to construct query result sets, compute true response volumes, and generate the leakage observations used by the attacks.

3.3 Workload Generation

Experiments operate on streams of queries generated according to different workload patterns. A query corresponds to selecting a plaintext value and retrieving all records in the dataset that contain that value. The leakage oracle then exposes information derived from this query, including the response volume and the partition identifiers associated with returned records in the SEAL system.

Three workload patterns were used to study how query structure affects leakage:

- **Uniform workload:** Query values are sampled uniformly at random from the set of possible values. This serves as a baseline workload with no bias toward frequently occurring values.
- **Zipf-like workload:** Queries are biased toward a small set of frequently occurring values. A "hot" fraction of values receives the majority of queries, while the remaining queries are drawn from less frequent values. This pattern models skewed real-world access behavior.
- **Hot-set workload:** Queries are restricted to a fixed working set consisting of a small subset of values (for example, the most frequent values in the dataset). This pattern models scenarios in which applications repeatedly access a narrow portion of the database.

Query streams typically contained several thousand distinct queries, depending on the run configuration. Fixed random seeds were used during dataset generation, workload sampling, and attack evaluation to ensure reproducibility while allowing independent runs with different seeds.

3.4 Experimental Parameters

The experiments varied several parameters to study how performance and leakage change under different configurations.

The primary parameter controlling SEAL behavior is the leakage parameter α. Increasing α increases the number of sub-ORAM partitions and reduces the size of each ORAM instance. Experiments evaluated values such as $\alpha \in \{0,1,2,3,4\}$ for lightweight runs and extended ranges for larger configurations.

To evaluate mitigation strategies, experiments also varied the padding parameter x. When padding is enabled, response sizes are rounded up to the next power of x. Tested values included $x = 2,4,8$ (and occasionally larger values) as well as the baseline case with no padding.

Additional parameters controlled the structure of observation windows used in the attack analysis. For over-time experiments, attacker success was measured at predefined checkpoints such as 10, 100, 500, and 1000 observed queries. Session-reset experiments used session lengths such as 10, 50, and 100 queries to study how leakage accumulates within short observation windows.

3.5 Repeated Runs and Randomness Control

To reduce the impact of randomness, experiments were executed using multiple independent seeded configurations. Each seeded run used independent random seeds for dataset generation, workload sampling, and attacker tie-breaking behavior. Trends were verified across runs to ensure that observed results were consistent and not artifacts of a particular random seed.

For session-based experiments, multiple independent sessions were generated for each session length. Attack success was computed separately for each session and then aggregated using mean and standard deviation across sessions.

3.6 Performance Metrics

Performance was evaluated by executing actual ORAM accesses and measuring both runtime and bandwidth-related metrics. Total wall-clock runtime was recorded for each experiment run, and average time per access was derived from the total runtime and the number of operations performed.

Bandwidth was estimated using a proxy based on the number of buckets read and written during each access. The approximate data transferred per access was computed using the number of bucket operations multiplied by the bucket capacity and block size. The SEAL wrapper logs per-access statistics including bucket reads, bucket writes, and approximate bandwidth, allowing performance comparisons across different values of α.

3.7 Security Metrics

Security was evaluated using the two implemented leakage-abuse attacks: Query Recovery Success Rate (QRSR) and Database Recovery Success Rate (DRSR).

The Query Recovery Success Rate (QRSR) measures the fraction of queries for which the attacker correctly infers the plaintext query value. It is computed as the number of correctly guessed query values divided by the total number of observed queries.

The Database Recovery Success Rate (DRSR) measures how accurately the attacker can map encrypted tuples to their corresponding plaintext values. It is computed as the number of returned tuples correctly labeled by the attacker divided by the total number of tuples returned across the evaluated queries.

Both metrics were evaluated either at observation checkpoints or over the entire query stream depending on the experiment type.

3.8 Padding Overhead Measurement

To quantify the cost of padding mitigation, experiments measured both the true response size and the padded response size for each query. If a query returned s real records, the padded size p was computed as the next power of the chosen padding base x. The overhead factor was then calculated as the ratio of the average padded response size to the average real response size across queries. This metric captures the additional bandwidth introduced by padding.

Overall, this experimental framework allows the performance and leakage properties of Path ORAM and SEAL to be evaluated across multiple dimensions, including partition leakage, workload structure, observation time, and mitigation strategies.

---

4. Results

This should be the biggest section. Break it into subsections.

4.1 Performance: Path ORAM vs SEAL

We first evaluate the performance cost of Path ORAM and SEAL by measuring the average bandwidth required per access. Bandwidth is estimated using the number of buckets read and written during each operation and converting this to an approximate number of bytes transferred. This metric captures the dominant cost of ORAM operations, since each access requires reading and rewriting encrypted buckets along a tree path.

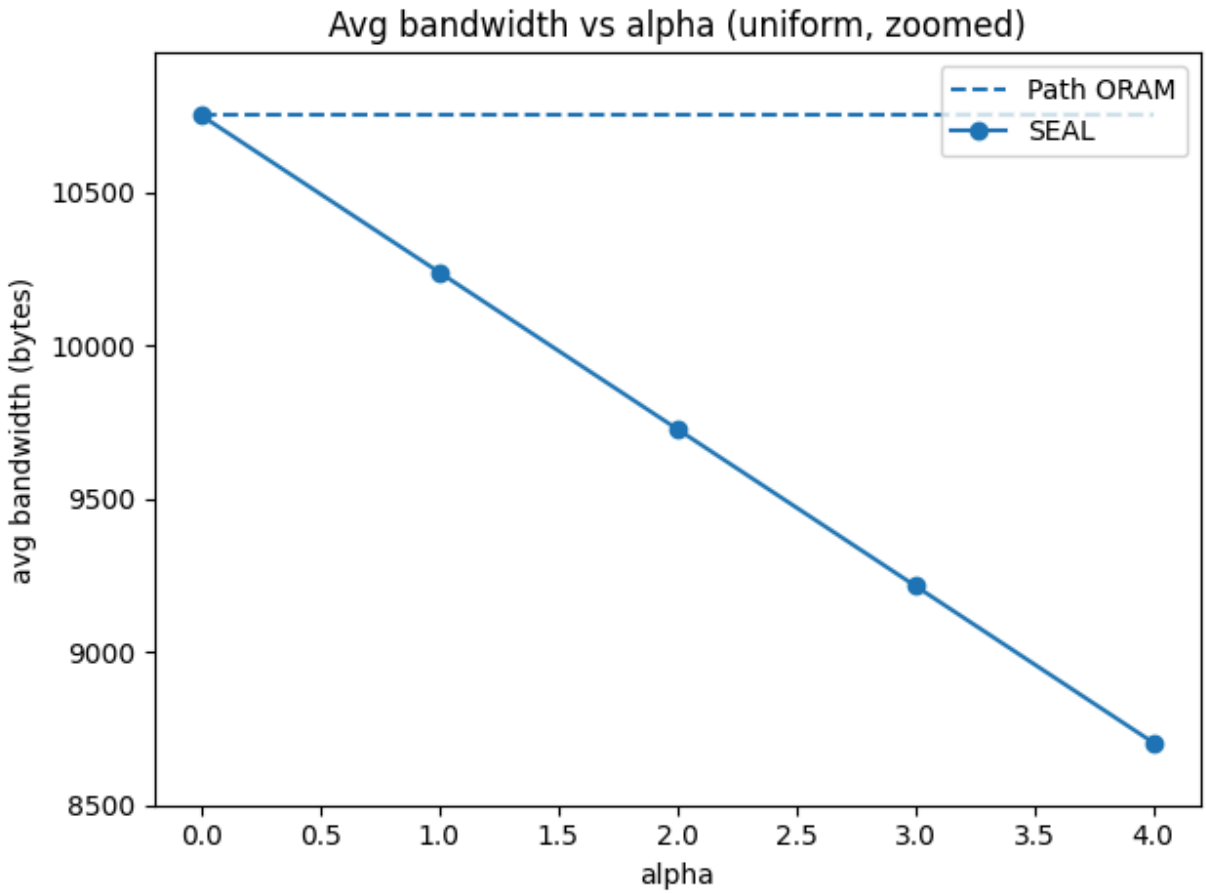

Figure 1: Average bandwidth per access as a function of the SEAL leakage parameter α under a uniform workload.

Figure 1 shows the average bandwidth per access as a function of the SEAL leakage parameter α under a uniform workload. The Path ORAM baseline is represented by the dashed horizontal line. As expected, Path ORAM exhibits a constant access cost because every operation reads and writes a full root-to-leaf path in a single ORAM tree.

In contrast, SEAL reduces the bandwidth cost as α increases. This occurs because SEAL partitions the database into 2^α smaller ORAM instances. Increasing α therefore reduces the size of each ORAM tree and shortens the path length that must be accessed. As a result, each access touches fewer buckets and transfers less data.

In our experiments, the average bandwidth per access decreases steadily as α increases from 0 to 4. While SEAL with α = 0 behaves similarly to the Path ORAM baseline, larger values of α produce noticeable reductions in bandwidth cost. At the largest tested value (α = 4), SEAL requires substantially less data movement per access than the baseline system.

We observed nearly identical performance trends across the other workload patterns (Zipf-like and hot-set). This is expected because the bandwidth cost of ORAM operations depends primarily on the tree structure and not on which logical blocks are accessed. Consequently, workload skew has little impact on the underlying access cost, while the SEAL parameter α directly controls the size of the ORAM structures and therefore the bandwidth required per operation.

Overall, these results confirm the expected performance advantage of SEAL. By increasing α, the system can significantly reduce the cost of each access, providing a clear efficiency benefit compared to the Path ORAM baseline.

4.2 Baseline Leakage vs α

We next evaluate how the leakage parameter α affects the success of the implemented leakage-abuse attacks. Security is measured using two metrics: Query Recovery Success Rate (QRSR) and Database Recovery Success Rate (DRSR). QRSR measures the fraction of queries for which the attacker correctly infers the plaintext query value, while DRSR measures the fraction of returned tuples that the attacker correctly maps to their plaintext values.

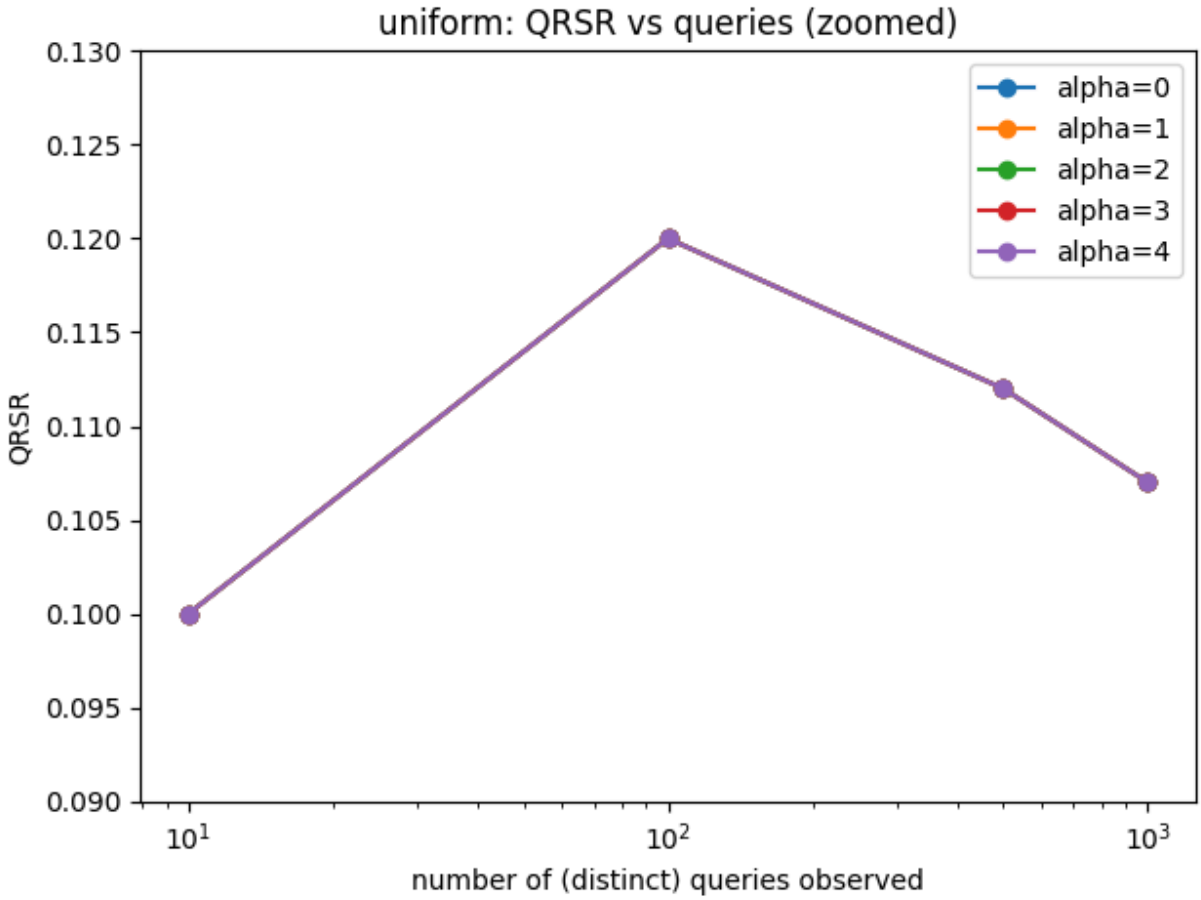

Figure 2: Query Recovery Success Rate (QRSR) as a function of the number of observed queries for different values of α under a uniform workload.

Figure 2 shows QRSR as a function of the number of observed queries for different values of α under a uniform workload. Across all tested values of α, the curves are nearly identical and show little separation. QRSR increases initially as more queries are observed, reaching roughly 0.12 after around 100 observations, and then fluctuates slightly as the number of observations increases. The lack of separation between α values indicates that the query recovery attack is primarily driven by response volume rather than by the partition identifiers exposed by SEAL. Since the response size for a given query does not depend on α, increasing the number of partitions does not significantly affect the attacker's ability to infer query values in this attack model.

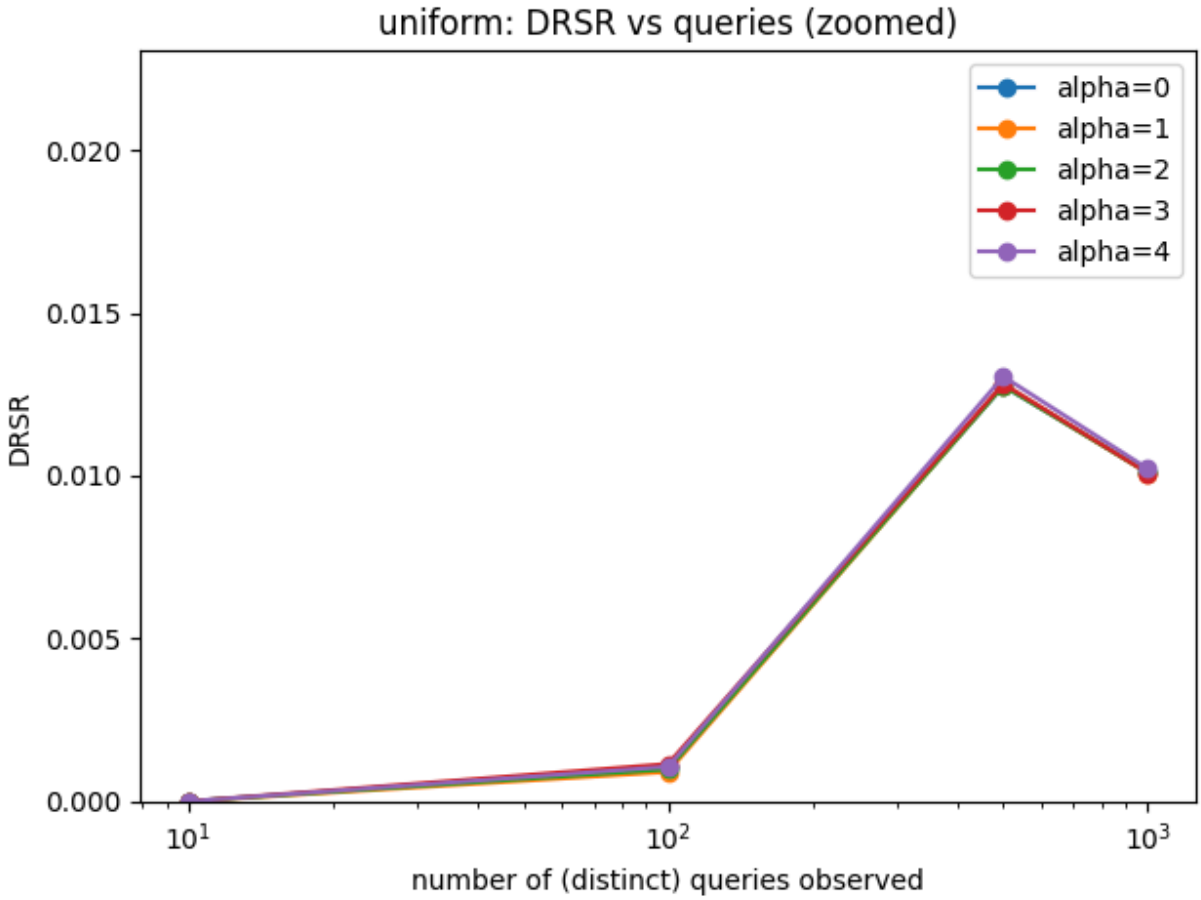

Figure 3: Database Recovery Success Rate (DRSR) as a function of the number of observed queries for different values of α under a uniform workload.

Figure 3 shows the corresponding results for the Database Recovery Success Rate (DRSR). Similar to QRSR, the DRSR curves for different α values largely overlap, with only minor differences across runs. DRSR remains very low when only a small number of queries are observed and increases gradually as more queries are processed, reaching approximately 0.01 after several hundred observations. This behavior reflects the difficulty of mapping individual encrypted tuples when query recovery itself is uncertain and multiple tuples share the same partition identifiers.

These results suggest that in this experimental configuration, the dominant source of information for the attacker is response volume rather than the α-bit partition identifiers leaked by SEAL. Because the implemented attacks rely heavily on volume information, varying α provides only modest additional signals for the attacker to exploit.

It is also important to note that the number of observed queries in these experiments is relatively small (10–1000 observations). With longer observation traces, the attacker might accumulate additional information that could make the partition identifiers more useful for distinguishing between candidate mappings. However, within the tested range of observations, the additional

leakage introduced by larger α values appears to have limited impact on the success of the attacks.

Overall, these baseline results indicate that while SEAL exposes partition information controlled by α, the practical effectiveness of attacks in this setting is largely determined by response volume leakage rather than the partition identifiers themselves.

4.3 Effect of Access Pattern

We next examine how the query access pattern affects the effectiveness of the leakage-abuse attacks. In addition to the uniform workload used in the baseline experiments, we evaluate two additional query distributions: a Zipf-like workload, where a small number of values are significantly more frequent than others, and a hot-set workload, where all queries are drawn from a small working set of highly popular values.

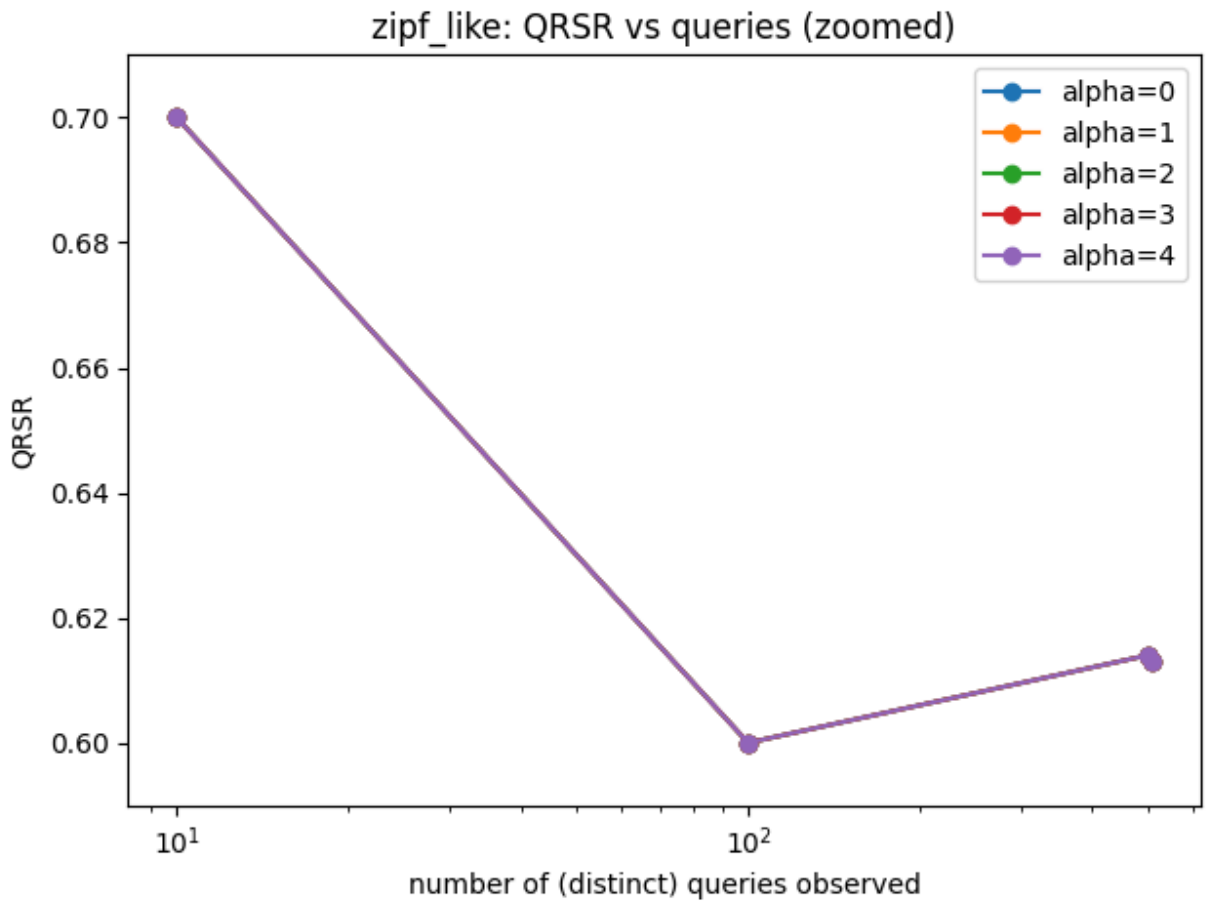

Figure 4: QRSR vs number of observed queries for the Zipf-like workload.

Figure 4 shows the Query Recovery Success Rate (QRSR) for the Zipf-like workload. In contrast to the uniform workload discussed in the previous section, the attacker achieves substantially higher success rates. Even with a small number of observed queries, QRSR reaches values around 0.6–0.7, indicating that the attacker is able to infer a large fraction of query values correctly. This improvement occurs because skewed workloads create stronger statistical signals in the observed response volumes. When certain values occur much more frequently than others, the attacker can more easily match observed response sizes to candidate plaintext queries.

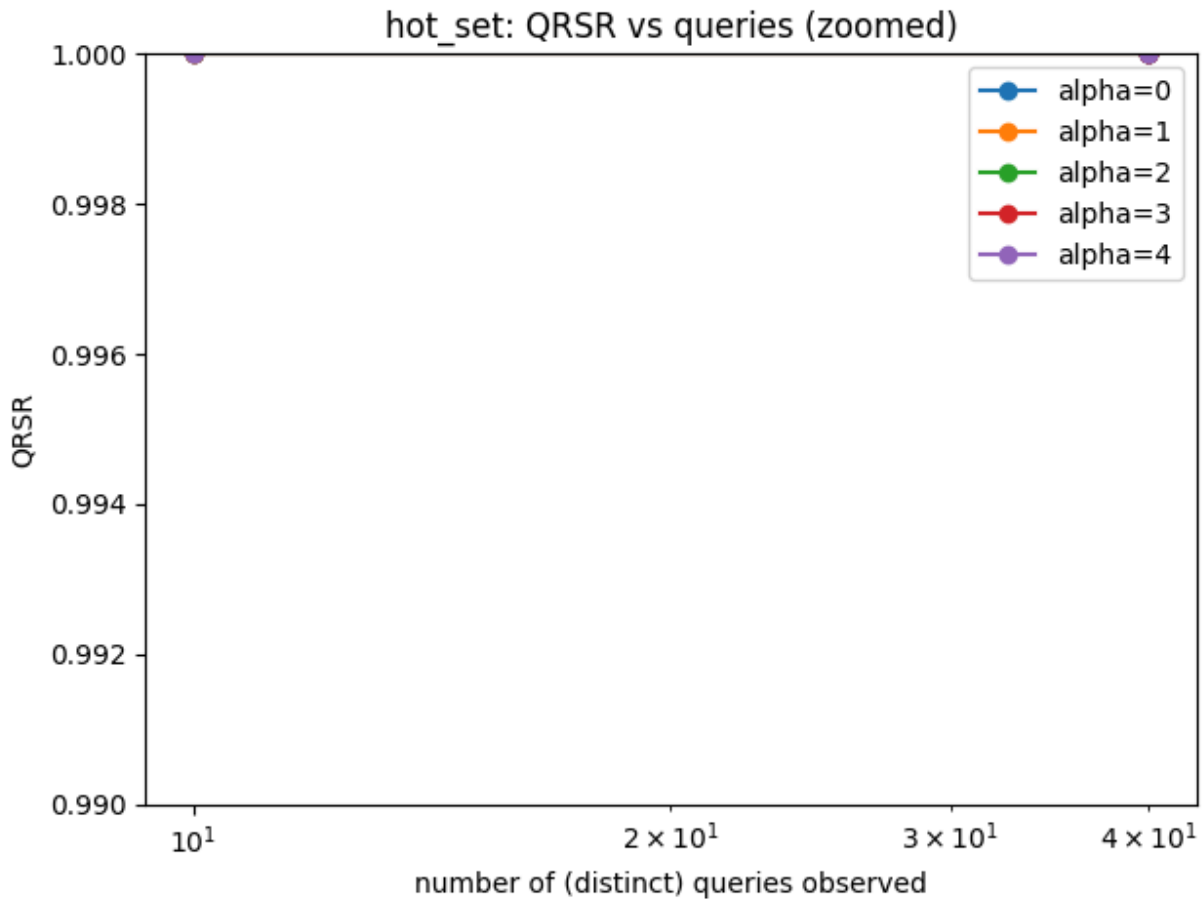

Figure 5: QRSR vs number of observed queries for the hot-set workload.

The effect is even more pronounced in the hot-set workload. As shown in **Figure 5**, QRSR remains essentially equal to 1.0 across all tested observation counts. In this setting, all queries are drawn from a very small set of frequently occurring values, making it extremely easy for the attacker to distinguish between candidate queries based on their response volumes. Because the possible query values are heavily restricted, the attacker can almost always identify the correct query after observing only a small number of responses.

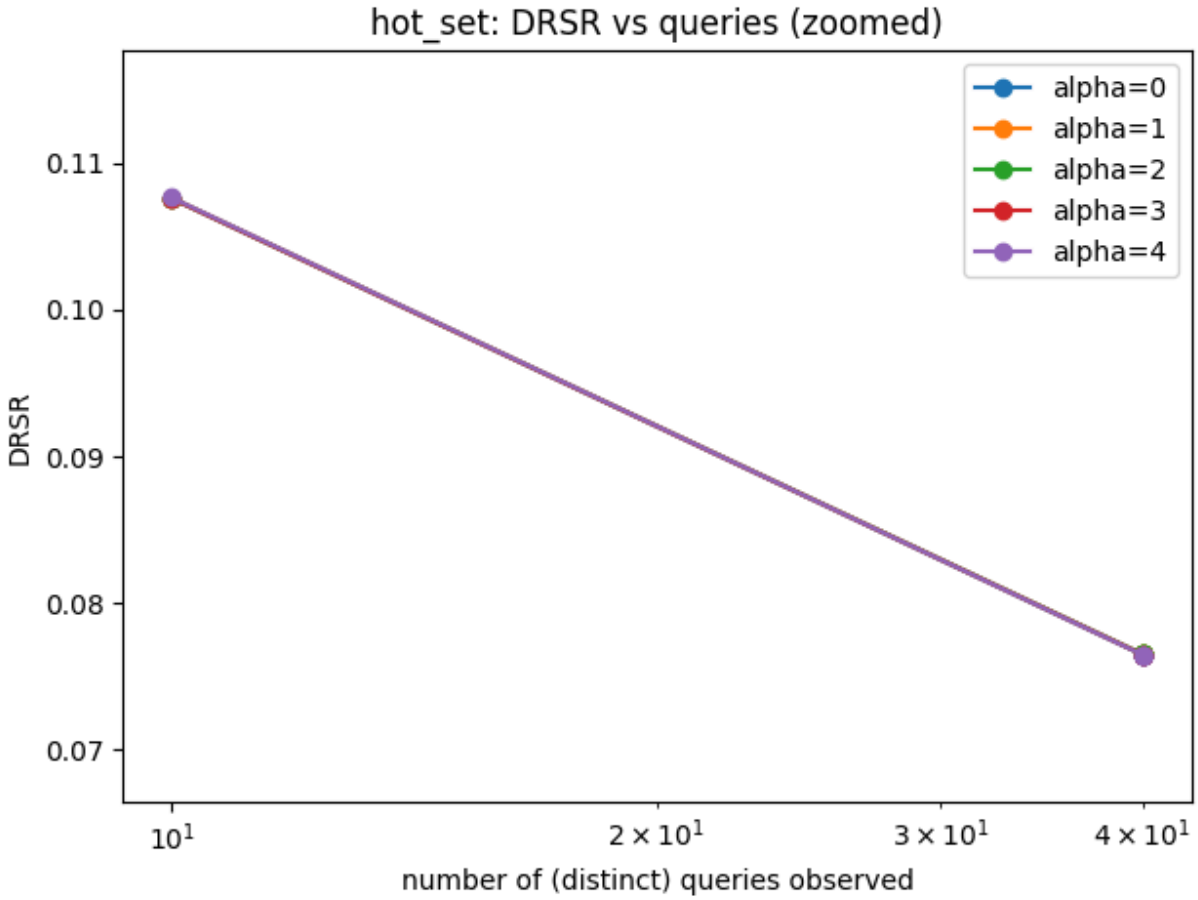

Figure 6: DRSR vs number of observed queries for the hot-set workload.

The corresponding DRSR results for the hot-set workload are shown in **Figure 6**. Under this workload, DRSR initially reaches relatively high values compared to the uniform case but decreases slightly as additional queries are observed. This behavior likely reflects the ambiguity that arises when multiple tuples share similar identifiers or response patterns within the limited working set. Even though individual queries are recovered with very high accuracy, reconstructing the entire database remains more challenging due to these overlapping response patterns.

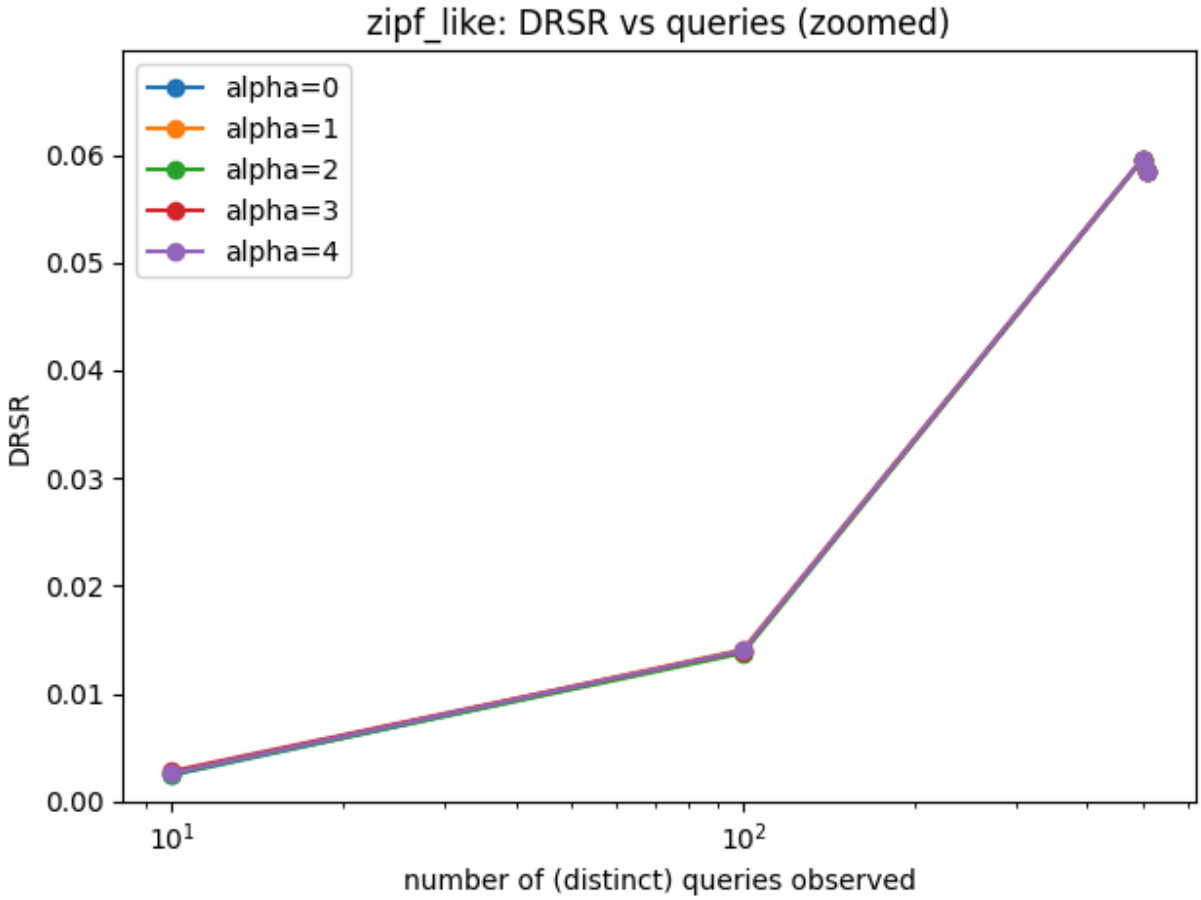

Figure 7: DRSR vs number of observed queries for the Zipf-like workload.

For comparison, the DRSR results for the Zipf-like workload are shown in **Figure 7**. In this case, DRSR increases steadily as more observations are collected, eventually reaching approximately 0.06 after several hundred observed queries. This trend reflects the moderate skew present in the Zipf distribution: while some values appear much more frequently than others, the overall query space remains large enough that the attacker requires more observations to reconstruct a significant portion of the database.

Importantly, across all workloads the curves corresponding to different α\alphaα values remain nearly identical. This reinforces the earlier observation that the primary driver of attack success in these experiments is response volume leakage rather than the partition identifiers introduced by SEAL. However, the workload distribution has a dramatic effect on how informative those volumes are. Highly skewed workloads amplify leakage by creating distinct and predictable response sizes for frequently accessed values.

Overall, these results highlight that workload characteristics can have a larger impact on attack effectiveness than the SEAL leakage parameter itself. Systems operating on skewed query distributions may therefore be significantly more vulnerable to leakage-abuse attacks than those with uniform access patterns.

4.4 Mitigation via Response Padding

To reduce the effectiveness of leakage-abuse attacks, we evaluate a simple mitigation strategy based on response padding. Instead of returning the exact number of records that match a query, the system pads each response to the next power of a chosen base *x*. This approach reduces the amount of information revealed by response volumes while introducing additional bandwidth overhead.

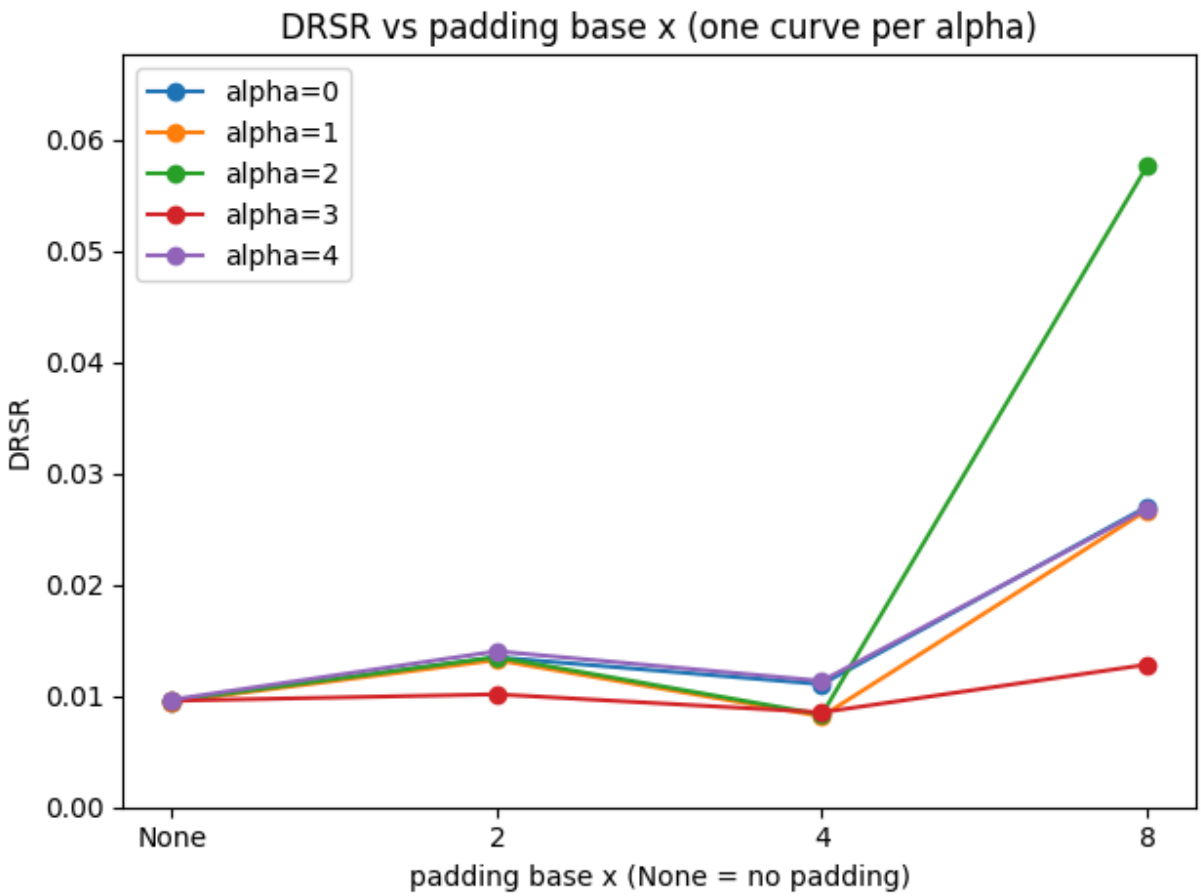

Figure 8: Database Recovery Success Rate as a function of the padding base x.

Figure 8 shows the Database Recovery Success Rate (DRSR) as a function of the padding base *x*. The value *x* = *None* corresponds to the baseline system without padding. As padding is introduced, the attacker's success rate generally decreases. In particular, when larger padding bases are used (e.g., *x* = 8), the DRSR drops noticeably compared to the unpadded baseline. This occurs because padding groups many different response sizes into the same padded value, reducing the attacker's ability to distinguish between candidate plaintext queries based on their observed volumes.

The curves corresponding to different values of α again remain very similar, indicating that the padding mitigation affects all partition configurations in roughly the same way. Since padding directly modifies the response volume leakage, it reduces the primary signal used by the attacks regardless of how the underlying database is partitioned.

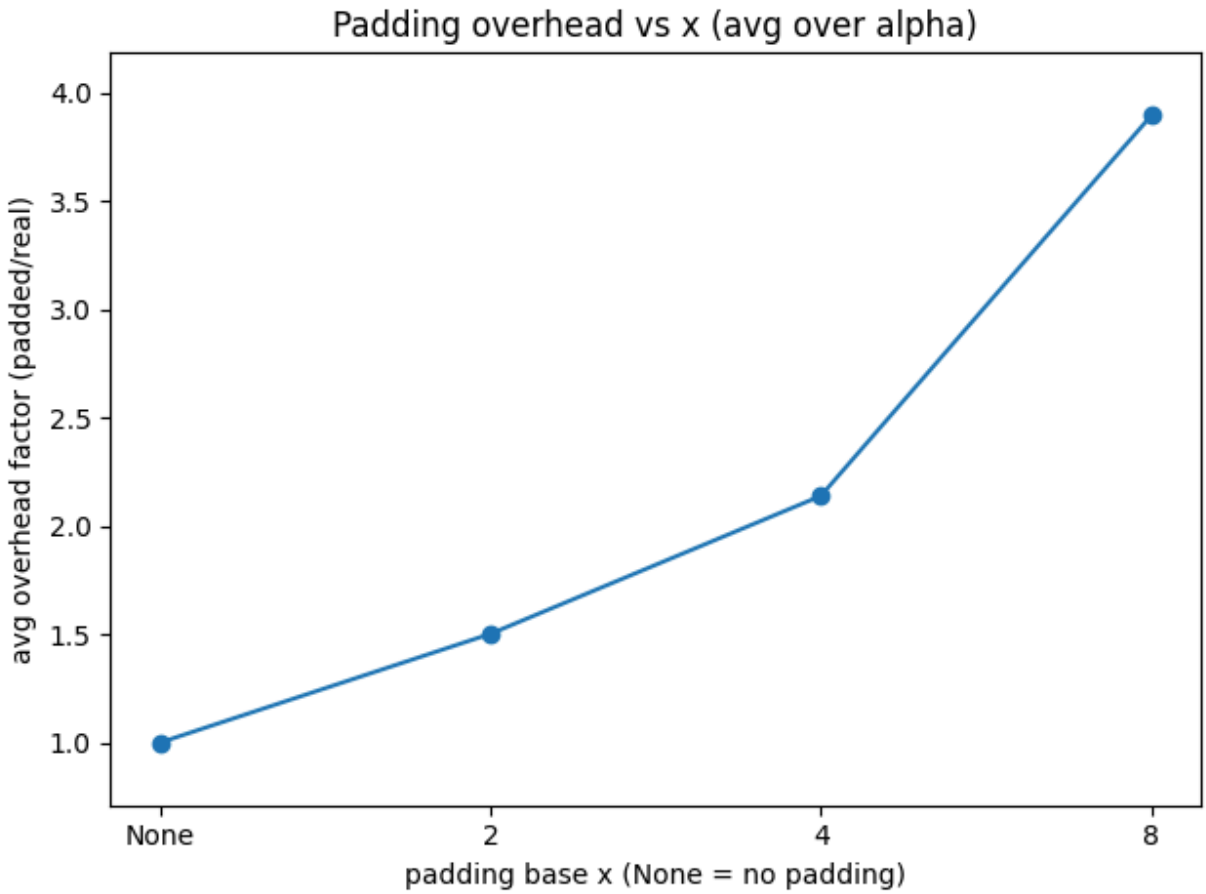

Figure 9: Average bandwidth overhead introduced by padding.

However, this improvement in security comes at a cost. **Figure 9** shows the average padding overhead factor, defined as the ratio between the padded response size and the true response size. As expected, the overhead increases with the padding base. When $x = 2$, the average overhead factor is approximately 1.5, meaning that responses are about 50% larger than necessary on average. For larger padding bases, the overhead grows significantly, reaching nearly a factor of four when $x = 8$.

These results illustrate the central tradeoff introduced by padding-based mitigations. Increasing the padding base reduces the information leaked through response sizes and can weaken the effectiveness of volume-based attacks. However, stronger padding also increases the amount of data transferred during each query, potentially impacting system performance and bandwidth consumption.

Overall, padding provides a simple but effective mechanism for reducing leakage, though its practical deployment requires carefully balancing the security benefits against the associated bandwidth overhead.

4.5 Session Reset Experiments

In addition to padding, we evaluate a mitigation strategy based on periodically resetting the attacker's observations. The intuition behind this approach is that leakage-abuse attacks often rely on accumulating information across a long sequence of queries. By limiting the number of queries that can be observed in a single session, the system may reduce the attacker's ability to correlate responses and gradually recover query mappings.

To study this effect, we evaluate attacker success as a function of the session length *L*, defined as the number of distinct queries observed before the attacker's knowledge is reset. For each session length, multiple independent sessions were simulated and the attacker's success rates were averaged.

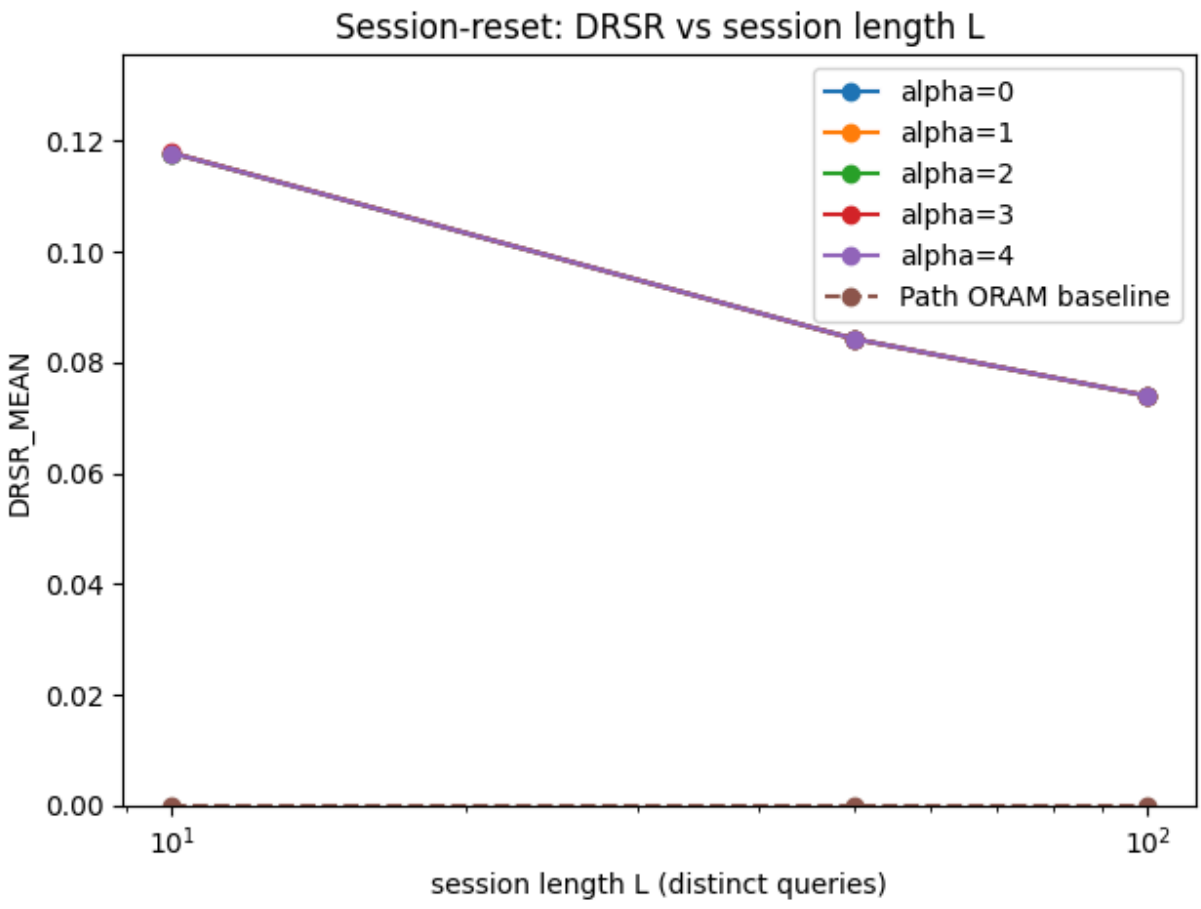

Figure 10: Mean Database Recovery Success Rate as a function of session length L.

Figure 10 shows the mean Database Recovery Success Rate (DRSR) as a function of the session length. As the session length increases from 10 to 100 queries, the attacker's ability to correctly map tuples decreases slightly. Shorter sessions allow the attacker to accumulate less information about the database, which limits the number of tuples that can be correctly identified. However, the overall change in DRSR is relatively modest across the tested range of session lengths.

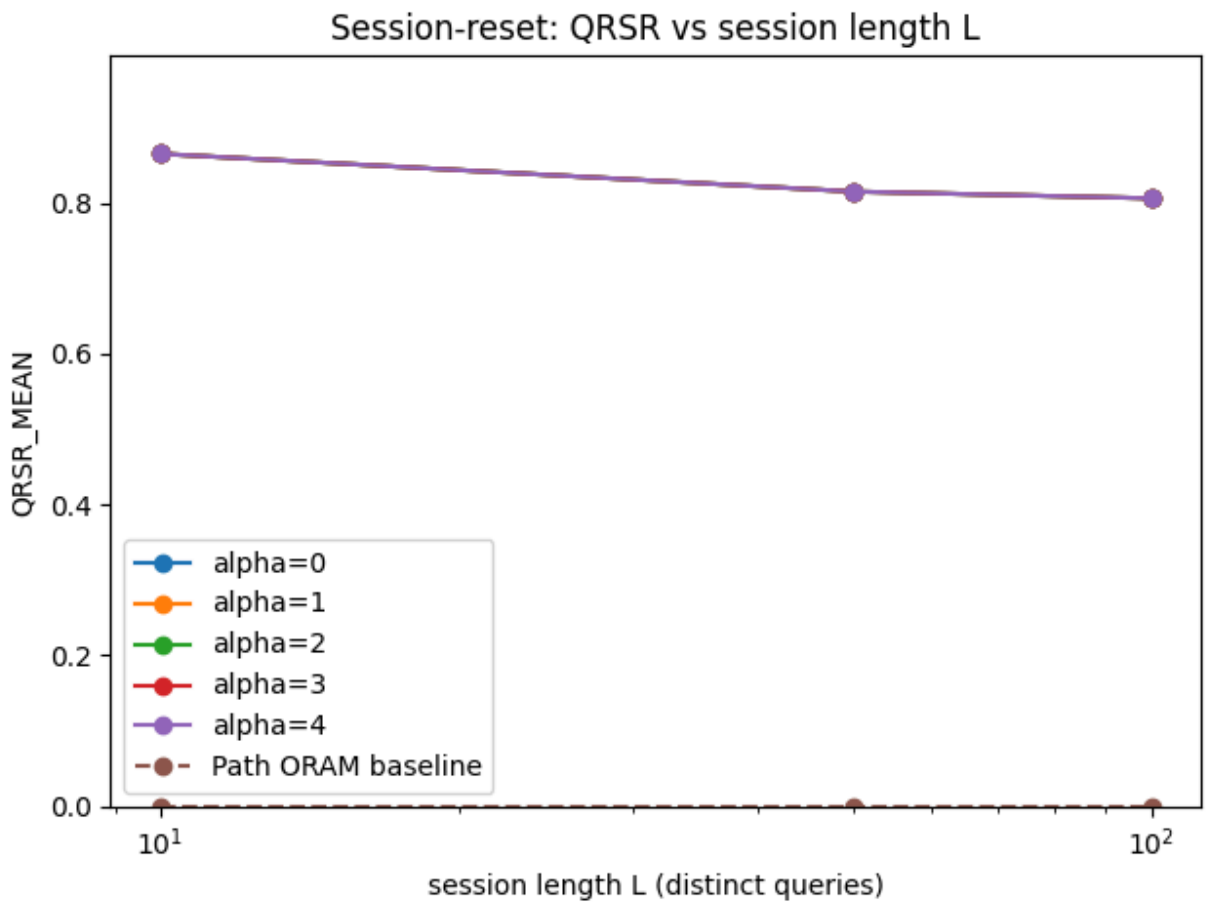

Figure 11: Mean Query Recovery Success Rate as a function of session length L.

Figure 11 shows the corresponding Query Recovery Success Rate (QRSR) results. Similar to the DRSR experiment, QRSR decreases slightly as the session length increases, but remains relatively high overall. This indicates that even with shorter observation windows, the attacker can still recover a substantial fraction of query values in skewed workloads. As in previous experiments, the curves corresponding to different values of α are nearly identical, reinforcing the earlier observation that the partition identifiers exposed by SEAL contribute little additional signal compared to response volumes.

For comparison, the Path ORAM baseline is also shown in the figures. Because Path ORAM hides response volumes entirely, the attacker obtains essentially no useful information in this setting, resulting in near-zero success rates. This highlights the fundamental tradeoff between

the stronger security guarantees of Path ORAM and the improved performance offered by SEAL.

Overall, these results suggest that session resets alone provide only limited protection against leakage-abuse attacks in the evaluated setting. While restricting the observation window slightly reduces attacker success, the underlying response volume leakage still provides sufficient information for the attacker to make accurate inferences.

4.6 Summary of Findings

Across all experiments, several key patterns emerge regarding the performance–security tradeoffs between Path ORAM and the SEAL extension.

First, the performance evaluation confirms the expected benefit of SEAL's partitioning approach. As the leakage parameter α increases, the database is divided into a larger number of smaller ORAM trees, reducing the path length that must be accessed for each query. This results in a consistent decrease in bandwidth per access compared to the Path ORAM baseline. The experiments therefore validate SEAL's primary design goal: improving performance by weakening the strict security guarantees of standard ORAM.

Second, the baseline leakage experiments show that varying α has relatively little impact on the success of the implemented attacks. Across multiple workloads, the curves for different α values largely overlap for both query recovery (QRSR) and database recovery (DRSR). This suggests that in the evaluated setting, the primary signal exploited by the attacker is response volume leakage, rather than the partition identifiers revealed by SEAL.

Third, the workload distribution has a substantial effect on attack effectiveness. Under uniform query distributions, attack success remains relatively low. However, when the workload becomes skewed—such as in the Zipf-like and hot-set patterns—the attacker's success rate increases dramatically. In particular, the hot-set workload leads to nearly perfect query recovery, highlighting how realistic access patterns can significantly amplify leakage.

Fourth, the padding mitigation experiments demonstrate that reducing the precision of response volumes can decrease the effectiveness of volume-based attacks. Larger padding bases reduce the attacker's ability to distinguish between queries, but this improvement comes at the cost of increased bandwidth overhead. The results illustrate the expected security–performance tradeoff: stronger padding provides better protection but significantly increases the amount of data transferred.

Finally, the session-reset experiments indicate that limiting the number of queries observed within a single session provides only modest protection against the attacks. Although shorter sessions slightly reduce attacker success, the underlying response volume leakage remains

sufficiently informative for the attacker to recover query information with high probability in skewed workloads.

Taken together, these findings highlight the central tradeoff explored in this project: improving efficiency through controlled leakage can significantly reduce system costs, but the practical security impact depends heavily on workload characteristics and the specific types of leakage that remain observable.

---

5. Discussion

The experimental results highlight several important insights about the practical security and performance tradeoffs of ORAM-based systems.

A central motivation behind SEAL is the observation that fully secure ORAM schemes, such as Path ORAM, can introduce substantial performance overhead due to the need to hide all access patterns. By allowing a controlled amount of leakage through database partitioning, SEAL reduces the cost of each access operation. Our experiments confirm this design intuition: increasing the leakage parameter α consistently decreases the bandwidth required per access, demonstrating that partitioning can significantly improve efficiency.

However, the results also show that the relationship between leakage and attack effectiveness is more nuanced than might initially be expected. Although SEAL explicitly reveals α-bit partition identifiers, our experiments suggest that these identifiers do not substantially increase the success of the implemented attacks. Instead, the dominant source of exploitable information appears to be response volume leakage. Because query result sizes are unaffected by the choice of α, the attacks achieve similar success rates across different partition configurations.

Another key observation is the strong influence of workload characteristics. While the attacks achieve relatively modest success under uniform query distributions, their effectiveness increases dramatically under skewed workloads. In particular, the hot-set workload produces nearly perfect query recovery in our experiments. This result highlights an important practical consideration: systems deployed in real-world environments often exhibit highly skewed access patterns, which may significantly amplify the risk of leakage-abuse attacks.

The mitigation experiments further illustrate the tradeoffs involved in protecting against these attacks. Response padding can reduce the information leaked through query result sizes, but doing so introduces additional bandwidth overhead. Larger padding bases provide stronger protection but can substantially increase the amount of data transferred during each query. Similarly, limiting the length of observation sessions slightly reduces attack effectiveness but does not eliminate the underlying leakage signal.

Finally, it is important to acknowledge several limitations of the experimental setup. The experiments were conducted on synthetic datasets and simulated workloads rather than real-world databases. Additionally, the number of observed queries in some experiments was relatively limited. In larger deployments or longer observation periods, attackers may be able to accumulate more information and achieve higher success rates. Future work could explore larger datasets, more diverse workloads, and additional mitigation strategies to better understand the practical security of systems that incorporate controlled leakage.

---

6. Conclusion

This project implemented and evaluated Path ORAM and the SEAL adjustable-leakage extension in order to study the tradeoff between performance and information leakage in encrypted database systems.

The experiments confirm that SEAL's partitioning approach can significantly reduce the bandwidth cost of ORAM operations, improving efficiency compared to a standard Path ORAM baseline. At the same time, the results demonstrate that even limited leakage can enable effective inference attacks, particularly when query workloads are skewed. In our evaluation, response volume leakage proved to be the dominant signal exploited by the attacker, while the additional partition identifiers exposed by SEAL had relatively little effect on attack success.

Mitigation techniques such as response padding can reduce the effectiveness of these attacks but introduce additional system overhead. Similarly, limiting observation sessions provides only partial protection when response volumes remain visible. These findings highlight the inherent tension between efficiency and security in practical encrypted data systems.

Overall, the results illustrate that systems designed with controlled leakage must carefully consider how different types of information interact with realistic workload patterns. While mechanisms such as SEAL can improve performance, understanding and mitigating the resulting leakage remains an important challenge for the secure deployment of encrypted databases.

---

7. References

[1] I. Demertzis, D. Papadopoulos, C. Papamanthou, and S. Shintre,

"SEAL: Attack Mitigation for Encrypted Databases via Adjustable Leakage,"

in Proceedings of the 29th USENIX Security Symposium (USENIX Security),

Boston, MA, USA, Aug. 2020.

[2] E. Stefanov, M. van Dijk, E. Shi, C. Fletcher, L. Ren, X. Yu, and S. Devadas,
"Path ORAM: An Extremely Simple Oblivious RAM Protocol,"
in Proceedings of the ACM Conference on Computer and Communications Security (CCS), 2013.

[3] S. Kellaris, G. Kollios, K. Nissim, and A. O'Neill,
"Generic Attacks on Secure Outsourced Databases,"
in Proceedings of the ACM Conference on Computer and Communications Security (CCS), 2016.